



Prioridades en Hilos



En Java, **todo hilo tiene una prioridad**, que el planificador *en teoría* podría usar para decidir qué hilo ejecutar antes. Pero Java es un lenguaje **generalista y portable**, así que **NO PODEMOS HACER SUPOSICIONES** sobre el comportamiento real del planificador

Explicación

Las prioridades siempre están en el rango **1 a 10**, definido por constantes:

```
Thread.MIN_PRIORITY // 1
Thread.NORM_PRIORITY // 5 (por defecto)
Thread.MAX_PRIORITY // 10
```

Cuando se crea un hilo nuevo:

- **hereda la prioridad del hilo creador**,
- y normalmente esa prioridad será **5**, porque el hilo `main` arranca con 5. Así que podemos decir que la prioridad por defecto de un hilo es 5.

Ejemplo:

```
Thread main = Thread.currentThread();
System.out.println(main.getPriority()); // imprime 5
```

¿Qué *debería* hacer el planificador?

En un mundo ideal:

- un hilo con prioridad 8 debería ejecutarse antes que uno con prioridad 5.

Pero Java se diseñó para funcionar igual en:

- Windows
- Linux
- macOS
- móviles
- servidores con 200 núcleos

y eso significa:

⚠ Java NO garantiza que un hilo de mayor prioridad se ejecute antes.

No se puede usar la prioridad para resolver problemas de concurrencia en esta asignatura.

Si necesitas asegurar alternancia o reparto del tiempo → **hazlo con sincronización**, no con prioridades.

Métodos importantes

setPriority(int)

Puedes cambiar la prioridad en cualquier momento:

```
hilo.setPriority(8);
```

Esto **no obliga** al planificador a nada.

getPriority()

Devuelve la prioridad actual.

yield()

Una sugerencia al planificador:

```
Thread.yield();
```

Es como levantar la mano y decir:

"Si quieres, ejecuta a otro"

Pero lo más común es que **no pase nada**. Puede incluso devolver el control *al mismo hilo*.

■ Nunca dependas de `yield()` para sincronizar nada.

```
public class PrioridadPorDefecto {
    public static void main(String[] args) {

        Thread creador = new Thread(() → {
            System.out.println("Creador → prioridad: " + Thread.currentThread().getPriority());

            Thread hijo = new Thread(() →
                System.out.println("Hijo → prioridad: " + Thread.currentThread().getPriority())
            );

            hijo.start();
        });

        creador.setPriority(8);
        creador.start();
    }
}
```

Hilos Daemon

Los **daemon threads** son hilos que realizan tareas en segundo plano:

- logs,
- limpieza de memoria,
- monitores,
- relojes internos...

La JVM **se cierra automáticamente cuando solo quedan hilos daemon**.

Por ejemplo:

- Si todos los hilos "normales" han acabado,
- pero quedan hilos daemon vivos,
- **la JVM los mata y cierra el programa igualmente.**

Cómo marcar un hilo como daemon

⚠ **Debes llamar a `setDaemon(true)` antes de `start()`.**

```
Thread t = new Thread(() -> { /* ... */ });
t.setDaemon(true); // correcto
t.start();
```

Si lo haces después → excepción.

Ejemplo completo

```
public class DaemonBasico {
    public static void main(String[] args) throws InterruptedException {
        Thread daemon = new Thread(() -> {
            while (true) {
                System.out.println("Daemon trabajando...");
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    break;
                }
            }
        }, "Daemon-thread");

        daemon.setDaemon(true); // OBLIGATORIO antes del start
        daemon.start();

        Thread.sleep(2000); // El main trabaja un poco...
```

```
        System.out.println("Main termina. La JVM se cerrará y el daemon morirá.");  
    }  
}
```

Salida típica:

```
Daemon trabajando...  
Daemon trabajando...  
Daemon trabajando...  
Main termina. La JVM se cerrará y el daemon morirá.
```

Cuando el main acaba, la JVM mata el daemon.